PA Project

Author : Dan Hagen

1 Load

The Load instruction is constructed with ra as a base register, rd as the destination register, the intimidate as an Offset. The top bit of the opcode differentiates a Load Word instruction from a Load Byte instruction where 1 = Byte and 0 = Load. Rb is currently unused but in the future could be used as an extension to the intimidate to allow for larger offset values.

0	6	11	16	21 – 31
X01011	Base reg.	unused	Dst	Offset LO
opcode	*ra*	*rb*	*rd*	*imm*

Figure 1: 32-bit Instruction Format

1.1 Fetch

The PC is first sent to the branch predictor and ITLB. If the branch predictor finds no matching entry, no action is taken. The ITLB translates the PC in normal mode or forwards it unchanged in admin mode. The resulting address is then used to access the I-cache, which either fetches the instruction from memory or returns a cached copy. The instruction is latched and passed to the Decode stage.

1.2 Decode

The instruction enters the decode unit, where the instruction is broken down and control signals are generated (e.g., load, byte/word, write enable). The rename unit assigns a unique destination tag and allocates the instruction in the ROB. It also checks whether source registers are pending writes in the ROB and forwards this information to the hazard unit, which determines whether to stall or use bypassed values. Source operands are then selected accordingly.

1.3 Execute

In the execute stage, the instruction verifies that the branch prediction was correct and computes the effective address by adding the base register (or bypassed value) to the offset.

1.4 Memory

The computed address is sent to the DTLB for translation, with admin mode bypassing translation. If the address is invalid, an exception is raised, the pipeline is flushed, and the instruction is marked as an exception in the ROB. On a valid translation, the store buffer is checked for forwarding; otherwise, the D-cache is accessed to fetch the data.

1.5 Write Back

When the instruction reaches the head of the ROB, it is committed and the result is written to the architectural registers.

2 Store

The store instruction is build up similarly to the Load instruction with the change that rb now represents the source register and rd is unused. rb was chosen as the source register instead of rd to keep the forwarding consistent. Again in the future rd should be included to offset to increase the possible size of the offset value.

0	6	11	16	21 … 31
X01100	Base reg.	Source reg.	unused	Offset LO
opcode	*ra*	*rb*	*rd*	*imm*

Figure 2: 32-bit Instruction Format

2.1 Fetch

The instruction flows through the branch predictor Itlb and I cache in the same manner as the load instruction and gets latched into the Decode stage.

2.2 Decode

The instruction enters the decode module where the instruction again is broken down and the control signals are generated. The rename unit still creates a unique tag fro the instruction but does not rename any registers as the instruction doesn't enable writing. The hazard unit again chekcs for Read after Write conflicts regarding the rb and ra registers and determines whether a stall or bypassed is needed. Source operands are then selected accordingly with the aforementioned changes to the Offset composition.

2.3 Execute

In the execute stage, the instruction verifies that the branch prediction was correct and computes the effective address by adding the base register (or bypassed value) to the offset

2.4 Memory

Passing though the DTLB the effective address and save value go into the store buffer. In the store buffer each entry consists of a adress , data value and 4 bit byte mask. The byte mask indicates which bytes are valid for the specific store simplifying later forwarding and communication with the Dcache. Stores are save aligned so a cross word store (last 2 bit != 00) gets saved in two entries with the respective byte mask specifying the valid parts of each word. If the store buffer is full earlier stages get stalled until the store buffer is able to drain enough to accommodate for the new store entry.

2.5 Write Back

When the instruction reaches the head of the ROB, it is retired from the ROB.

3 Branch/Jump

The Branch instruction is build up using the ra and rb registers as the compare, rd as the branch type indicator (BEQ, BLT , BGT, jmp, jlx) and immediate as the offset. Th jump instruction uses the same setup but ra and rb are unused.

0	6	11	16	21 – 31
001101	Source reg.	Source reg.	Brn Type	Offset
opcode	*ra*	*rb*	*rd*	*imm*

Figure 3: 32-bit Instruction Format

3.1 Fetch

The instructions PC first enters the Branch Predictor here if a hit occures the following pc will be set to the saved target pc. On a miss the FIFO queue will be updated to "reserve" a space for this branch/jump instruction. Following this the PC again flows through the ITLB and Icache in the same manner as other instructions.

3.2 Decode

The instruction enters the decode module where the instruction again is broken down and the control signals are generated. Additionally to the opcode the rd field here defines the branch compare instruction. The rest of the Decode phase executes as usual.

3.3 Execute

In the Execute stage additionally to adding the Instructions PC to the Offset for a branch or the ra register value to the Offset for a jump to determine the target PC we also for brnanches compare the ra and rb register values to figure out the branching outcome. Both target pc and true branch outcome get compared to the branching prediction from the fetch stage. If both match the prediction was correct and we can continue with out any flushing. If the prediction however was false we flush the Decode and Fetch stages and set the PC to the correct target pc, either the Branch/Jump target pc or Branch PC + 4 depending on branch outcome)

3.4 Memory

As the instruction does not memory access it doesnt do anything in this stage.

3.5 Write Back

Similar to the memory stage, again this instruction doesnt write anything so has barely anything to do in this stage except for committing and advancing the ROB head pointer.

4 Arithmetic

The Arithmetic instruction are build up using the ra and rb registers as the source registers, rd as destination register and the immediate is only used for the addi instruction replacing one of the souurce registers as the second term in the arithmetic operation.

0	6	11	16	21 — 31
00XXXX	Source reg.	Source reg.	Dst. reg.	Imd
opcode	*ra*	*rb*	*rd*	*imm*

Figure 4: 32-bit Instruction Format

4.1 Fetch

The Fetch stage executes the same as for all other instructions.

4.2 Decode

The instruction enters the decode module where the instruction again is broken down and the control signals are generated. Furthermore a 4 bit alu operatoion wire is set that defines the opartion to be performed int the alu unit. The rest of the Decode phase executes as usual.

4.3 Execute

In the Execution unit the alu operation defined by the alu op wire defined by the decoder is performed. If the instruction is however a multiplication it is instead forwarded to the Multiplication stage which separately handles all multiplication operation. If this unit already has a multiplication in flight (1 multiplication take 5 cycles in the Mul unit.) the Execution and earlier stages are stalled until the multiplication instruction can be latched into the unit.

4.4 Memory

As the instruction does not do any memory access it doesn't do anything in this stage. Here earlier multiplication instruction which earlier left the standard pipeline rejoin. If the instruction that normally would latch from this stage into the write back stage is a writing instruction earlier stages including this stage get stalled to make space for the multiplication otherwise it just latches into the wb stage.

4.5 Write Back

Here the instruction become ready in the ROB and if they are the head will commit to the registers.

5 Exceptions+ROB

5.1 Entry Structure

Each entry in the Reorder Buffer (ROB) follows the structure shown in Figure **??**. The fields are defined as follows:

- **V (Valid):** Indicates whether the ROB entry contains a valid in-flight instruction.
- **R (Ready):** Indicates whether the instruction has completed execution and its result is available for use or forwarding.
- **WE (Write Enable):** Specifies whether the instruction writes a result back to the architectural register file.
- **IRET:** Indicates that the entry corresponds to an `IRET` instruction.
- **EXC (Exception):** Indicates that an exception was detected during the execution of the instruction.
- **RD (Destination Register):** Specifies the architectural destination register that will be written on commit.
- **VALUE / EXC_PC:** Stores the computed result of the instruction. If **EXC** is asserted, this field instead contains the program counter (PC) of the faulting instruction.

V (1b)	R (1b)	WE (1b)	IRET (1b)	EXC (1b)	RD (5b)
VALUE or EXC_PC [31:0]					

Figure 5: Reorder Buffer (ROB) entry format.

5.2 Allocation

When a instruction enter the decode stage its destination register gets assigned a tag. This tag basically indexes the most recent entry of the destination register in the ROB. It basically acts as a pointer to the ROB entry/tag currently holding the newest not-yet-committed value for that register. Its allocated in the rob at that tag with the corresponding entry bits set accordingly. None write back instructions are directly set to ready.

5.3 Commit

When a write back instruction reaches the WB stage the ROB updates its entry, with the corresponding value and sets its Ready bit to 1. If the instruction is also the head of the ROB it gets committed otherwise it stays in the ROB until the head pointer reaches the entry. This ensure instructions are committed to the registers in order. Any entry with ready bit set to 1 commits when reaching the head of ROB reaching the WB is not mandatory. This for example allows Mul instructions to overwrite none write back instructions when squeezing into the WB stage with out any stalls required.

5.4 Exceptions

If a exception is detected in any stage the ROB gets the informed by the exception unit and marks the corresponding entry as an excption and saves its PC in the Value field. If the exception happens before the instruction is allocated in the ROB, so in the F or D stage the information gets floped into the EX stage where the Exception unit now catches it. Once a exception is marked in the ROB, earlier pipeline stages are flushed and stalled until the entry commits. On commit the entry and earlier entries are Flushed out of the ROB as well and renaming is set back accordingly. The PC gets set to a hard coded Exception pc and admin bit is set to 1.

6 Performance Results

6.1 Buffer Sum

Instructions: 652
Cycles: 1337
CPI = 2.050613

For the buffer sum test, the CPU achieves the above results. Looking at the waveform, we can see two main culprits responsible for slowing down the pipeline.

First is the I-cache, which periodically stalls to fetch new instructions. This is likely due to the small cache size evicting previously loaded instructions before branching back to the beginning of the loop. The CPU currently employs prefetching, which allows the I-cache to load the next instruction line while the current instruction line is being used. This aims to prevent stalling every four instructions, but in cases like the buffer sum, where a small loop is heavily utilized, combined with the small cache size, this behavior can create unnecessary stalls by increasing the rate at which I-cache entries cycle through the cache.

The second major source of stalling is a D-cache miss on load instructions, specifically the following load instruction:

```
2c802800    // [VA PC 36] load  r4  r0  r5   0        # r5 = a[i]
```

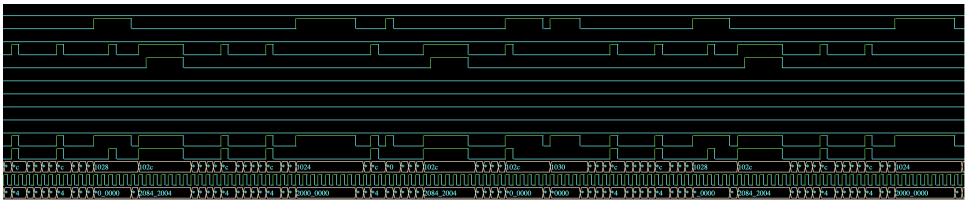

Figure 6: Buffer Sum waveform

6.2 Memory Copy

Total:
Cycles = 1275
Instructions = 656
CPI = 1.943598

For the memory copy test, we observe a slightly better CPI. Looking at the waveform, however, we see that it suffers from similar issues as the Buffer Sum. In this case, the D-cache miss and I-cache miss partially overlap, allowing both to be handled simultaneously and thus decreasing the overall stall time.

Once again, a larger I-cache or a more robust replacement policy than FIFO would greatly improve performance, as loop instructions that are repeated frequently throughout execution would no longer cause recurring stalls.

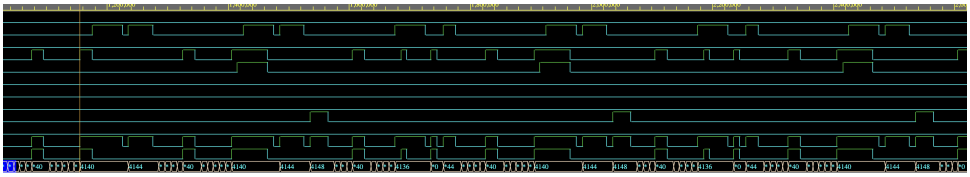

Figure 7: Memory Copy waveform

6.3 Matrix

Instructions: 12845850
Cycles: 57,824181
CPI = 4.501639

For the multiply test, the CPU achieves the above results. Looking at the waveform, we can observe similar pipeline behavior to the previous benchmarks. T

As with the previous tests, the I-cache periodically stalls when fetching new instructions, particularly due to the limited cache capacity. This results in frequently evicted instructions when branching back to the start of the loop. Increasing the Icache size by 4 lines already reduces the CPI to around 3.5.

Additionally, D-cache stalls occur during operand loads required for the multiply operations, further contributing to the overall CPI.

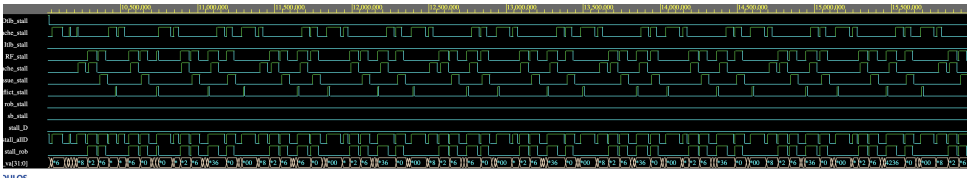

Figure 8: Multiply waveform